you set up a configuration watching thousands of files, you could run into difficulties!

In listing 10.5, any changes you make to the file will be mirrored in the `IConfiguration`. But as I said at the start of this chapter, `IConfiguration` isn't the preferred way to pass settings around in your application. Instead, as you'll see in section 10.3, you should favor strongly typed objects.

10.3 Using strongly typed settings with the options pattern

In this section you'll learn about strongly typed configuration and the options pattern, the preferred way of accessing configuration in ASP.NET Core. By using strongly typed configuration, you can avoid problems with typos when accessing configuration. It also makes classes easier to test, as you can use simple POCO objects for configuration instead of relying on the `IConfiguration` abstraction.

Most of the examples I've shown so far have been about how to get values *into* `IConfiguration`, as opposed to how to *use* them. You've seen that you can access a key by using the `builder.Configuration["key"]` dictionary syntax, but using `string` keys this way feels messy and prone to typos, and the value retrieved is always a `string`, so you often need to convert it to another type. Instead, ASP.NET Core promotes the use of strongly typed settings—POCO objects that you define and create and that represent a small collection of settings, scoped to a single feature in your app.

The following listing shows the map settings for your store locator component and display settings to customize the home page of the app. They're separated into two different objects with `"MapSettings"` and